# Reinforcement Learning with the Acrobot, comparison of different model-free approaches

*Odysseas Georgios Konias [el19166]*

# The acrobot

The acrobot is a planar robot consisting of two links connected to form a chain. The one end of the chain is fixed and the joint between the two links is actuated. It is essentially a robotic arm with actuation at the elbow but no actuation at the shoulder. Its name is derived from its similarity to an acrobat swinging from a bar, who mostly controls his motion from the waist rather than the wrist.

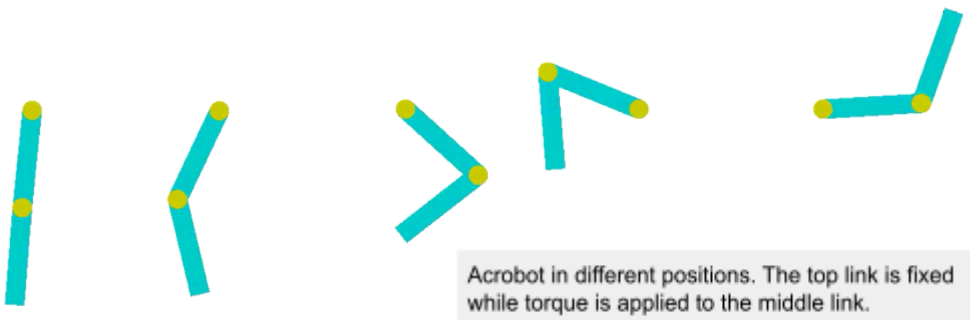
The most common task for the acrobot is to get the robot from the resting to the inverted pendulum position. This can be divided into two challenges: the swing up problem where the goal is to bring the bottom end above a certain height and the balancing problem where the robot has to maintain itself in the inverted pendulum position. We will be focusing on the swing up problem.

## The environment

The environment is quite complex. The robot has 4 degrees of freedom—angles and angular velocities of both the first and second links—and only one actuator which is constrained on the torque it can apply. The dynamics of the robot are non linear due to the nature of the inverted pendulum. In order to complete the tasks the controller has to take into account the coupling between the actuated degree of freedom and the unactuated degree of freedom. This poses a significant challenge for classic control methods. Instead, we turn to Reinforcement Learning (RL) which learns through interaction with the system and try model-free methods that require no prior knowledge of its internal dynamics.

## Simulation in Gymnasium

We will use Open AI's Gymnasium (commonly known as Gym) to simulate the swing-up problem. Gymnasium is an open source python library for developing and comparing RL algorithms with a diverse collection of environments. Specifically we will work with the acrobot environment "Acrobot-v1".



Acrobot in different positions. The top link is fixed while torque is applied to the middle link.

In Gym, the agent starts with an observation of the environment, selects an action and then takes a step in the environment to receive the next observation. During each step, the agent also receives a reward representing whether he did good or bad and through which it learns. The agent continues taking actions until the episode is finished (an episode is a complete run of the game).

In the acrobot environment:

- **Observation**: The agent receives an ndarray with 6 values providing information about the two joint angles and their angular velocities (sin, cos, and angular velocity for each link).
- **Actions**: The agent's actions are discrete and denote the torque applied to the actuated joint. Possible values are -1, 0, or +1.
- **Reward System**: If the agent fails to reach the goal during a step, it receives a reward of -1. Successfully reaching the target height awards a reward of 0 and ends the episode.

The action space is very large, we have 6 continuous variables to represent the state of the acrobot. Also notice that rewards are very sparse, the agent does not get information on whether it is doing well or not until the very end of the episode. This is particularly challenging when the agent is not yet trained since the end of the episode takes many steps (taking random actions ends the game on average in about 2200 steps).

## The RL methods

We will test four RL methods on the swing up problem: Monte Carlo, SARSA, Deep Q-Networks (DQN) and Policy gradients. They are all model free methods meaning they do not rely on any prior knowledge of the model's dynamics.

The first three define a policy by estimating the values of state-action pairs. The **value function** ([1],[9]) indicates how good it is for an agent to be in a specific state. If a model of the environment is available the agent simply chooses the action that leads to the state with the highest value. If a model is not available state values are not sufficient for an agent to determine a policy. It is then useful to estimate action values (values of state-action pairs) rather than state values and pick the action with the largest value thus determining a policy. The value function of the state action pairs is also known as a **Quality function Q** [9]. Monte Carlo, SARSA, and DQN all attempt to do this in different ways.

Policy gradients is a different approach that directly defines a **policy function** which decides which action to take [9]. The policy is parameterized and optimized using gradient descent. This results in a stochastic policy where actions are chosen based on probabilities derived from this policy distribution.

# Monte Carlo

Monte Carlo methods are the most basic model free learning methods for estimating value functions and discovering optimal policies. These methods aim to estimate the value of a state, defined as the expected return starting from that state. The monte carlo approach is to simply average all the returns observed after visiting that state.

In control problems, the focus shifts to state-action values. Here, for each state-action pair, we average the returns observed after taking that action. Once we have these state-action value estimates we can derive a policy by selecting the action with the highest state-action value for any given state.

## Key concepts for RL algorithms (For more details, refer to [1])

**Cumulative reward G:**
The cumulative reward G is the discounted sum of all returns from a specific time t to the end of an episode. It is calculated as:
$G = \gamma G + r$ , where γ is a discount factor ranging from 0 to 1, and r is the reward observed when taking an action. Each time a reward is received, the value of G is updated accordingly.
For each state-action pair, we maintain a list of returns. Each time we encounter the pair, we append G to this list. The state-action value is then the average of this list of returns.

**Generalized Policy Iteration:**
In order to find an optimal policy we use Generalized Policy Iteration (GPI). In GPI we maintain an approximate policy and an approximate value function. The action-values are improved through monte carlo prediction and the policy updated to select actions with the highest state-action values.

**Maintaining exploration, exploring starts, ε-soft policies**
For any reinforcement learning algorithm to converge to an optimal policy it must ensure continuous exploration. That is we have to guarantee that all actions can be selected infinitely often. One simple method is **exploring starts**, where every state-action pair has a non-zero probability of being chosen as a starting point in each episode. However, this is not always feasible, as seen in specific cases like the acrobot. An **ε-soft policy** is a policy where each action is selected with a probability of at least ε, ensuring that even suboptimal actions are occasionally chosen, which promotes thorough exploration of the action space. Among them we usually use **ε-greedy policies** where the best known action is selected most of the time, but with a small probability ε, a random action is chosen.

**On policy vs Off policy**
In on policy methods the method that is used to produce an episode is the same as the policy that is being improved. In off policy methods we can have two policies, the behavior policy which produces the episode and the target policy which is the one being optimized.

## The algorithm:

The algorithm used is from [1].

**On-policy first-visit MC control (for $\varepsilon$-soft policies), estimates $\pi \approx \pi_*$**

Algorithm parameter: small $\varepsilon > 0$
Initialize:
  $\pi \leftarrow$ an arbitrary $\varepsilon$-soft policy
  $Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
  $Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):
  Generate an episode following $\pi$: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless the pair $S_t, A_t$ appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
      Append $G$ to $Returns(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
      $A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)
      For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

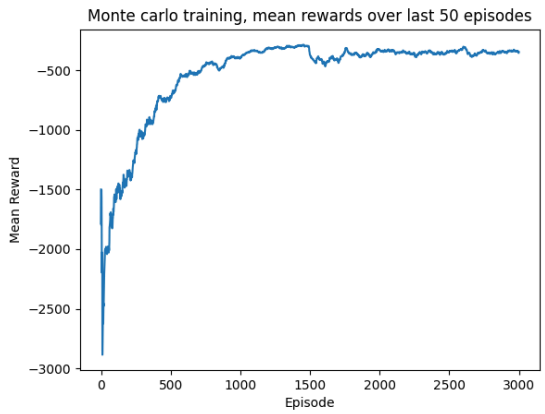
## Implementation details:

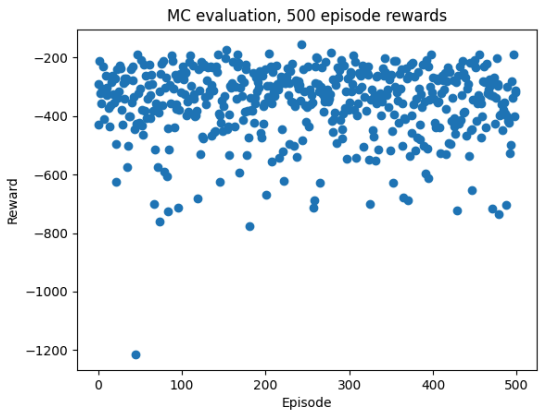
In order to store state-action values we must first discretize the state. The state consists of 6 float numbers, for each one we will make a discrete variable with 10 points. Thus the Q values are stored in a table of size 10^6*3(1million states and 3 actions) and is initialized with all values at 0.
The policy is an ε-greedy policy with ε starting at 1 and ending at 0.01. Thus the policy is at first ε-soft and gets greedier over time.

## Results:

We evaluate by running 500 games with the algorithm, always picking the action with the highest q value, and getting the mean score. For comparison, selecting actions randomly completes the game on average on 2200 steps (-2200 reward).

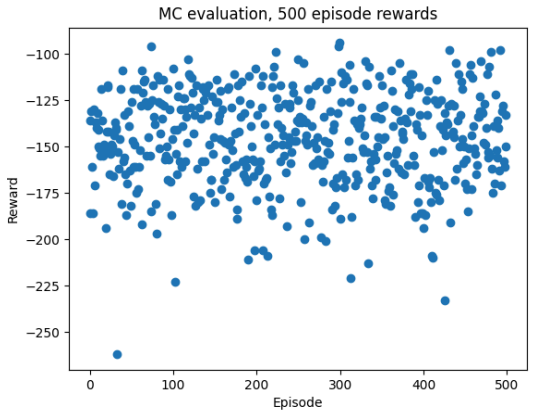
**Training for 3000 games:** (Elapsed time 394 seconds)



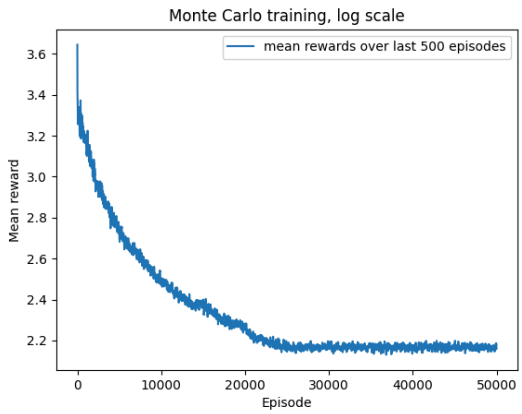

Score: 354

**Training for 50000 games:**




**The agent gets a score of around 140**

We plot the training curve on a logarithmic scale. Since the values are negative, we use their absolute values. Additionally, we smooth the data by averaging over 100 episodes.




The training curve shows that performance plateaus before the end of training, indicating that further training will not significantly improve the agent's performance. It is worth noting that the epsilon value starts at 1 and decreases to 0.01 by the midpoint of training. This gradual reduction means the agent doesn't always make optimal decisions initially, impacting the reward in each episode.

**Conclusion:**

While the Monte Carlo algorithm is simple, it effectively estimates the Q function, allowing the agent to solve the problem. However this approach requires extensive training. Training for 50000 episodes takes hours to complete.

One major issue is that it only updates at the end of each episode, making it difficult to cover the large acrobot state space. This also results in very slow training because the episodes are initially long and only shorten as the algorithm learns.

Another problem is that the algorithm treats all actions as contributing equally to the overall reward. In an environment where episodes take thousands of steps to be completed this can be problematic as the effect of some actions can be lost.

# Sarsa:

Sarsa is a **temporal difference (TD) learning** algorithm. In contrast to monte carlo learning TD learning updates its value function estimates continuously throughout an episode, adjusting at each step based on the received rewards.
The simplest TD method called TD(0) makes the update:

$$V(S_t) \leftarrow V(S_t) + \alpha\Big[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\Big]$$

when transitioning from the state $S_t$ to $S_{t+1}$ and receiving reward $R_{t+1}$
We update the estimate of the value function $V(S_t)$ adding the difference between the new estimate $R_{t+1} + \gamma V(S_{t+1})$ and the old estimate $V(S_t)$. Alpha is a learning variable and γ is the discount factor.
The idea is based on the **Bellman equation:**

$$V(\mathbf{s}) = \max_{\boldsymbol{\pi}} \mathbb{E}\left(\mathbf{r}_0 + \gamma V(\mathbf{s}')\right)$$

where $s' = s_{k+1}$ is the next state after $s = s_k$, $r_0$ is the received reward, and the expectation is over actions selected from the optimal policy.

Similarly to TD(0), to learn the Q function on-policy we use the **SARSA (state-action-reward-state-action)** algorithm. The update equation for sarsa is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big],$$

when transitioning from one state action pair to the next $(S_t, A_t) \rightarrow (S_{t+1}, A_{t+1})$ and receiving reward $R_{t+1}$. Again we update by adding the difference between the new estimate $R_{t+1}+\gamma Q(S_{t+1}, A_{t+1})$ and the old estimate $Q(S_t, A_t)$.
(For more details refer to[1]).

## The algorithm:

The algorithm used is from [1]

**Sarsa (on-policy TD control) for estimating $Q \approx q_*$**

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize $S$
  Choose $A$ from $S$ using policy derived from $Q$ (e.g., $\varepsilon$-greedy)
  Loop for each step of episode:
    Take action $A$, observe $R$, $S'$
    Choose $A'$ from $S'$ using policy derived from $Q$ (e.g., $\varepsilon$-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until $S$ is terminal

## Implementation details:

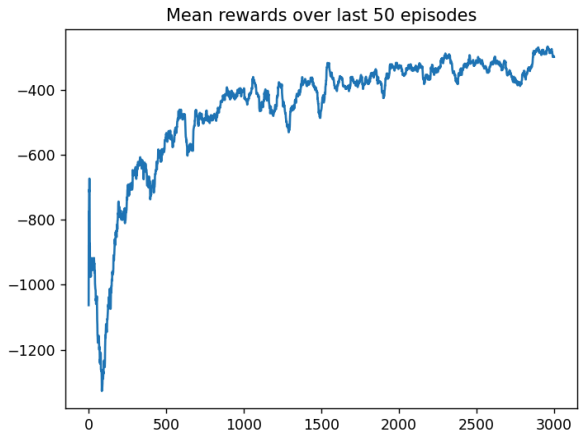
The implementation is from [2].
The state is discretized in the same way as in the monte carlo algorithm. The Q values are stored in a table of size 10^6*3(1million states and 3 actions) and is initialized with all values at 0. We again use an e-greedy approach with e starting at 1 (random) and ending at 0.1. The learning rate a is set to 0.1 and γ to 0.99.

## Results:

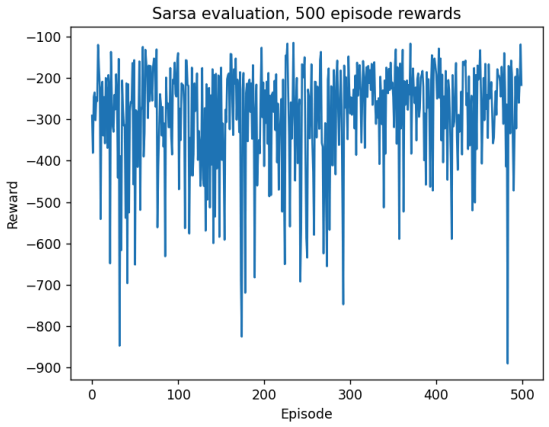
We evaluate by running the agent for 500 episodes and getting the mean score.

**Training for 3000 episodes: (**Elapsed time: 318 seconds)

Training

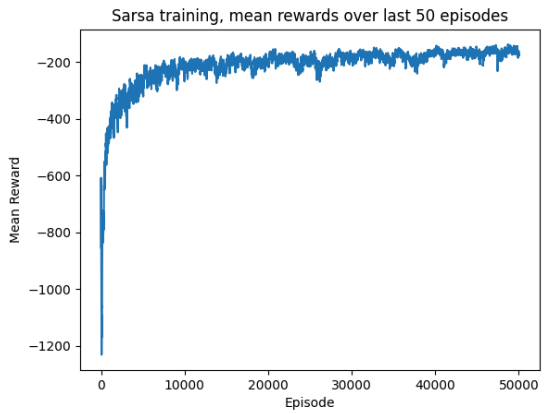


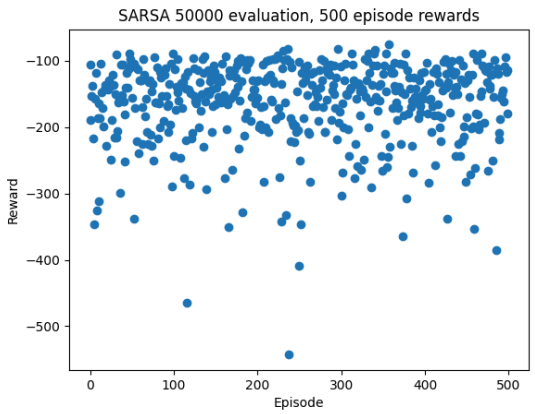

Evaluation




After 3,000 episodes of training, the agent achieved a score of approximately 300.

**Training for 50000 episodes:**



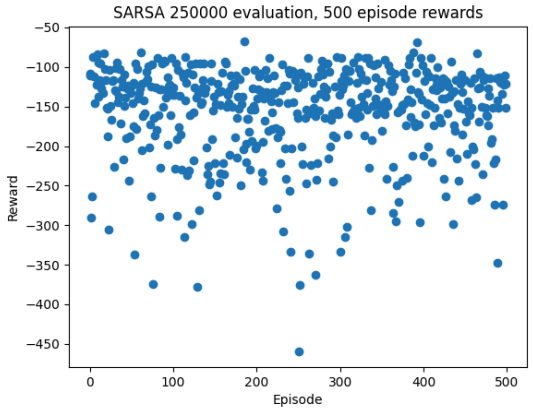




**Score of around 170**

**Training for 250000 episodes:**

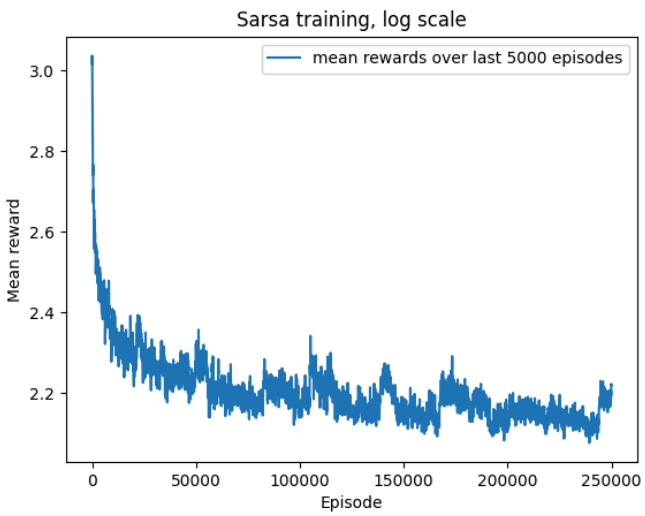
Training

Evaluation:




Score of around 155

We plot the training curve on a logarithmic scale. Since the values are negative, we use their absolute values. Additionally, we smooth the data by averaging over 100 episodes.




The plot shows that the reward keeps decreasing until the end, indicating that further training may be beneficial for the agent.

**Conclusion:**

Even though SARSA converges faster, since it is trained throughout an episode, it has overall worse performance than Monte Carlo. This could be because long-term associations matter more than immediate rewards in the acrobot environment. The best solution would likely be a TD(n) approach, which combines the two methods by updating the Q matrix throughout the episode, based on both immediate and long-term rewards. However, we will not be examining this and will instead move on to a more complex approach: Deep Q Learning.

# DQN:

In Monte Carlo and Sarsa methods, a table is used to store and update the Q values. An alternative approach is to use a function Q, where for each state-action pair (s, a), Q(s, a) returns the corresponding Q value. The Deep Q Learning approach is to try to learn such a function using a neural network.

The **Deep Q-Networks (DQN)** algorithm was invented by Mnih et al.[3]. The problem is a case of unsupervised learning as we do not have any labels indicating the correct values of the Q function. Thus the model has to learn from unsupervised data.

We will use two networks. The first called the **policy network** is used to estimate the Q-values for the current state s and action a: Q(s,a|θ) where θ is its weight vector. The second called the **target network** has the same architecture and is used to estimate the Q values of the next state s' and action a'. Only the policy network's weights are updated using backpropagation and thus all the learning takes place in the policy network. The policy network's weights are copied to the target network periodically.

**Replay buffer:**
The training algorithm is an off policy method that gathers training data while running, saves it on a buffer and then uses it to train the network. This buffer is called the replay buffer. Specifically the agent selects the next action based on the predicted Q values and receives the next state and a reward. Those are then stored in the replay buffer as pairs e.g. (State, action, reward, next_State) to be used as training data instances.

**Loss:**
For the loss we first calculate the difference between the expected Q value and the predicted Q value using the **Bellman equation:**

$$V(\mathbf{s}) = \max_{\boldsymbol{\pi}} \mathbb{E}\left(\mathbf{r}_0 + \gamma V(\mathbf{s}')\right)$$

where

- The policy network predicts the Q values for state s and the one corresponding to the action a is selected.
- The target network predicts the Q values for the next state s' from which we select the maximum
- r is the reward, (s, a, r, s' are the values from the replay buffer)

This difference is then fed into the loss function e.g. quadratic loss
For more details on the approach refer to [4], [5].

## Training:

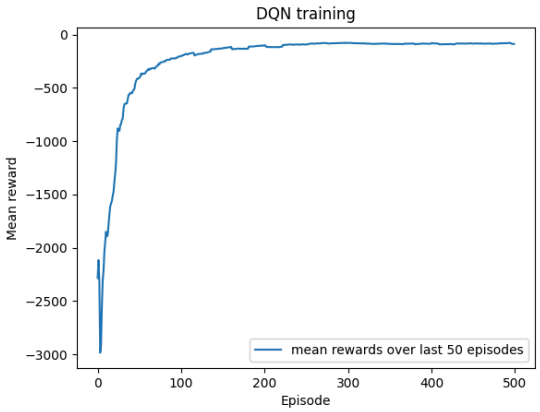
In each episode the agent starts from a random state and selects the next action based on the Q values predicted by the policy network. It executes an ε-greedy action and receives the next state and reward which are then stored in the replay buffer. After every step we also execute a training step, where we calculate the loss function and perform backpropagation to update the policy networks weights.

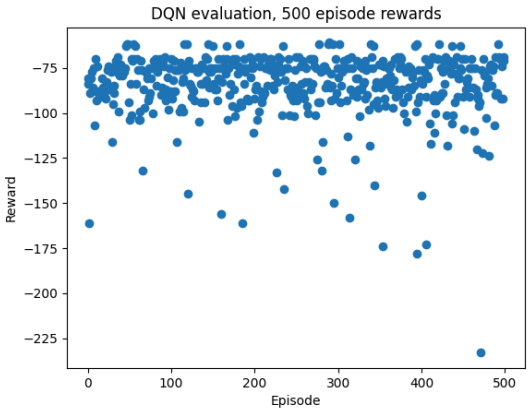
## Implementation details:

The implementation used is from [5]. The architecture for the network includes two linear layers of size 128. To ensure the agent is exploratory an ε-greedy algorithm is used with ε starting at 1 and reaching 0.01 during training. The target network's weights are updated slowly over time to match those of the policy network.
The learning rate is set to 1e-4, gamma is set to 0.99 and the batch size is 128. The update rate of the target network, tau, is set to 0.005.

## Results:

We train the model for 500 episodes: (Elapsed time: 378 seconds)




**The model gets a score of around -85**

**Conclusions:**

The score is significantly improved compared to agents trained with monte carlo and sarsa, and achieved through significantly less training (fewer games played). Using a function for the Q values eliminates the need to discretize the state. In Monte Carlo and SARSA there was a limit to the complexity of the model due to the large amounts of memory we need to properly discretise the state space. With DQN the only limit is the complexity of the model. However, training a DQN requires careful tuning of the hyperparameters.

We could possibly achieve even better results by using different network architectures. While experimenting with the hyperparameters we tried using a network with higher dimension hidden layers which did not yield better results. One could try using a completely different neural network architecture.

# Policy Gradients:

The algorithms we covered, Monte Carlo, Sarsa and DQN are all value-based algorithms. They all approximate the value function Q and derive a policy by selecting the action with the highest Q value. An alternate approach is to directly define a **policy function** which decides which action to take. Unlike methods that estimate the value of a state or a state action pair, the policy function directly estimates the probability of taking action α from state s: π(s|a) thereby defining policy π. This approach results in a stochastic policy, as opposed to previous deterministic policies that always select the action with the highest score [6].

The optimal policy is the one that achieves the highest possible return in a finite trajectory, so our goal is to maximize the expected return. With the policy gradients algorithm we will directly parameterise the policy $\pi_\theta$ and adjust the parameters θ using gradient ascent in the direction that maximizes the expectation of the return [6].

Our policy function will be implemented using a neural network with a parameter vector θ. The network takes the state as input and outputs the probabilities of taking each action. By sampling the output distribution we will select the next action.

**Performing gradient ascent:**
We wish to maximize the expected return defined as:

$$J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{\tau\sim\pi_\theta}[\sum_{t=0}^{T} r(s_t, a_t)] = \int \pi_\theta(\tau)R(\tau)d\tau$$

We will move the parameters in the direction of the gradient of the expected return:

$$\theta_{k+1} = \theta_k + \alpha\nabla_\theta J(\pi_\theta)$$

We need to derive an expression for the expected return that we can calculate. We take the following steps [7]:

**Derivation for Basic Policy Gradient**

$$\begin{aligned}
\nabla_\theta J(\pi_\theta) &= \nabla_\theta \mathop{\mathrm{E}}_{\tau\sim\pi_\theta}[R(\tau)] \\
&= \nabla_\theta \int_\tau P(\tau|\theta)R(\tau) && \text{Expand expectation} \\
&= \int_\tau \nabla_\theta P(\tau|\theta)R(\tau) && \text{Bring gradient under integral} \\
&= \int_\tau P(\tau|\theta)\nabla_\theta \log P(\tau|\theta)R(\tau) && \text{Log-derivative trick} \\
&= \mathop{\mathrm{E}}_{\tau\sim\pi_\theta}[\nabla_\theta \log P(\tau|\theta)R(\tau)] && \text{Return to expectation form} \\
\therefore \nabla_\theta J(\pi_\theta) &= \mathop{\mathrm{E}}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)R(\tau)\right] && \text{Expression for grad-log-prob}
\end{aligned}$$

The final expression consists of the gradient of the log-prob of a trajectory and the trajectory's reward. We can estimate this expectation using a sample mean. If we collect a set of trajectories D using the policy $\pi_\theta$, we can estimate it with the moving average:

$$\hat{g} = \frac{1}{|\mathcal{D}|}\sum_{\tau\in\mathcal{D}}\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)R(\tau)$$

In our case we can define a loss function as:

$$L(\theta) = -\mathbb{E}_{\tau \sim \pi_\theta}[\sum_{t=0}^{T} \pi_\theta(a_t|s_t)R(\tau)]$$

and calculate it by taking many trajectories and calculating the mean of the sum.

## Training:

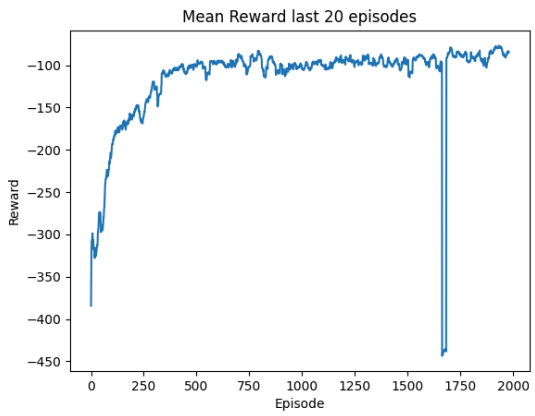
In each episode the agent plays the game according to the policy $\pi_\theta$ defined by the network and collects the trajectory (states, actions followed) and rewards of each episode. After enough data has been collected (denoted by the hyperparameter steps_per_train_step) we proceed to optimize the model. This involves computing the loss using the collected data and performing backpropagation.

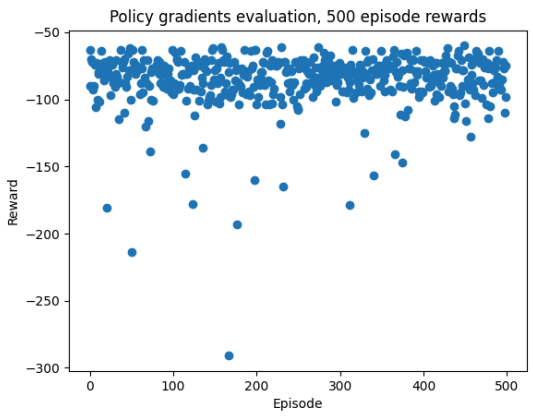
## Implementation details:

The implementation is based on [6]. The network architecture consists of two linear layers, with the hidden layer having 32 units. The network outputs a probability distribution over three possible actions, from which one action is sampled during each step. The training process runs for 2000 episodes, and the network undergoes a training step after every 1000 steps of collected data.

## Results:

The rewards during training: (Elapsed time: 129 seconds)




We evaluate by running 500 episodes. **The agent gets a mean reward of around -87**

### Conclusions:

The agent achieves similar results to the DQN. Despite training for more episodes, Policy gradients training was faster likely due to the simpler network architecture. It is also a more straightforward approach requiring only one network and no replay buffer.

# Conclusions:

The Acrobot environment is classified as moderately complex due to its nonlinear dynamics, characteristic of the double pendulum system, which complicates the accurate prediction of future states. The state space, comprising six continuous variables, is fairly large. Additionally, the reward structure for the swing-up problem is sparse, providing rewards only upon task completion, thereby making training more challenging.

Despite these challenges, all four algorithms tested are effective in solving the swing-up problem. The first 2 methods ,despite their simplicity, successfully tackled the task but required extensive training. Initially, SARSA was expected to outperform since it is a more sophisticated approach; however, Monte Carlo achieved better results. This is likely due to the sparsity of the reward system where immediate rewards don't matter as much as long term rewards.

The latter 2 methods, utilizing neural networks, surpassed Monte Carlo and SARSA, achieving much better performance with much less training. Both achieved similar results and must be tested on harder tasks to be compared further. For simple tasks policy gradients appears to be more straightforward and easy to implement.

Future research could compare these methods in terms of energy efficiency for actuation. Additionally, exploring other approaches, such as the actor-critic method that integrates both value and policy functions, could potentially enhance performance. Furthermore, future work could test these algorithms on the balancing problem and the combined swing-up and balancing task, which represents the complete control of the Acrobot.

# References:


1. R. S. Sutton and A. G. Barto, "Monte Carlo Methods and Temporal Difference Learning," in *Reinforcement Learning: An Introduction*, 2nd ed. [Online]. Available: http://incompleteideas.net/book/RLbook2020.pdf, pp. 91-117, 119-158.
2. P. Tabor, "Acrobot.py," *GitHub*. [Online]. Available: https://github.com/philtabor/Youtube-Code-Repository/blob/master/ReinforcementLearning/Fundamentals/acrobot.py.
3. V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529-533, Feb. 2015. [Online]. Available: LETTER - Human-level control through deep reinforcement learning
4. F. Shang, "Reinforcement learning explained visually (part 5): Deep Q-networks step by step," *Towards Data Science*, November 30, 2021. [Online]. Available: https://towardsdatascience.com/reinforcement-learning-explained-visually-part-5-deep-q-networks-step-by-step-5a5317197f4b.
5. F. Enendu, "Implement a deep reinforcement learning method for an acrobot game," *Medium*, August 21, 2021. [Online]. Available: https://medium.com/@enendufrankc/implement-a-deep-reinforcement-learning-method-for-an-acrobot-game-f1f71b1aead2.
6. M. Sanz, "Introduction to reinforcement learning part 5: Policy gradient algorithms," *Medium*, June 14, 2021. [Online]. Available: https://markelsanz14.medium.com/introduction-to-reinforcement-learning-part-5-policy-gradient-algorithms-862960f7b0dc.
7. OpenAI, "Part 3: Intro to policy optimization," *Spinning Up in Deep Reinforcement Learning*. [Online]. Available: https://spinningup.openai.com/en/latest/spinningup/rl_intro3.html#part-3-intro-to-policy-optimization.
8. Farama Foundation, "Acrobot," *Gymnasium*. [Online]. Available: https://gymnasium.farama.org/environments/classic_control/acrobot/.
9. S. L. Brunton and J. N. Kutz, "Reinforcement Learning," in *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*, 2nd ed. Cambridge, UK: Cambridge University Press, 2022, pp. 419-444.[Online]. Available: https://faculty.washington.edu/sbrunton/databookRL.pdf